

Reviewer Pack — Minimal Rank of Hodge Structure Bounds Monotone Circuit Size...

Entry 5b9dbc90b6a9 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-28 01:27:09 UTC

Minimal Rank of Hodge Structure Bounds Monotone Circuit Size for k-CLIQUE

Entry ID: 5b9dbc90b6a9 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-28 01:27:09 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Hodge Theory) **Field B** (complexity object): Complexity Theory: Boolean Circuit Monotonicity

Statement:

[‘For every n , the minimal rank of the Hodge structure associated with the incidence variety of a k -clique on an n -dimensional Boolean cube is $\Omega(n^2 \log k)$.’, ‘Furthermore, there exists a constant c such that for all sufficiently large n , the minimal rank of this Hodge structure is at most cn^2 .’, ‘This implies a lower bound on the size of monotone circuits for k -CLIQUE: any such circuit must have size $\Omega(n^2 \log k)$.’]

Rationale (proposer’s reasoning):

[‘Hodge theory provides a framework to study algebraic varieties, including those associated with combinatorial objects like graphs. The minimal rank of a Hodge structure is an invariant that captures the complexity of the variety.’, ‘Previous work has shown that the size of monotone circuits for k -CLIQUE grows exponentially with n and k , but no precise lower bound has been established using algebraic geometry techniques.’, ‘This conjecture would bridge these fields by providing a new method to derive circuit lower bounds using Hodge theory.’]

Taxonomy category: MONOTONE_CLIQUE (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): f34594b0631467fc

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, across all seeds, the mean minimal rank of Hodge structures for k -clique on Boolean cubes exceeds $n^2 \log k$ with a confidence interval not exceeding 5%.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.80	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Hodge structure" AND "k-clique" AND "Boolean cube" - "monotonicity in Boolean circuits" AND "lower bound" AND "Hodge theory" - "incidence variety" AND "circuit size" AND "algebraic geometry"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=1.8s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_k_clique(n, k):
        vertices = list(range(n))
        edges = []
        for _ in range(k):
            clique = random.sample(vertices, 2)
            if clique not in edges and (clique[1], clique[0]) not in edges:
                edges.append(clique)
        return vertices, edges

    def incidence_variety(vertices, edges):
        n = len(vertices)
        m = len(edges)
        incidence_matrix = [[0] * m for _ in range(n)]
        for i, (u, v) in enumerate(edges):
            incidence_matrix[u][i] = 1
            incidence_matrix[v][i] = 1
        return incidence_matrix

    def gaussian_elimination(matrix):
        n, m = len(matrix), len(matrix[0])
        rank = 0
        for i in range(n):
            if all(matrix[i][j] == 0 for j in range(m)):
```

```

        continue
    pivot_col = matrix[i].index(1)
    rank += 1
    for j in range(i + 1, n):
        if matrix[j][pivot_col] == 1:
            for k in range(m):
                matrix[j][k] ^= matrix[i][k]
    return rank

def min_rank_of_hodge_structure(n, k):
    vertices, edges = generate_k_clique(n, k)
    incidence_matrix = incidence_variety(vertices, edges)
    return gaussian_elimination(incidence_matrix)

n_values = [5, 10, 15, 20, 30, 40]
ranks = []
for n in n_values:
    for _ in range(5):
        rank = min_rank_of_hodge_structure(n, k)
        if rank is None:
            return {
                "metric_name": "min_rank",
                "metric_value": None,
                "instances_tested": 0,
                "conjecture_holds": False,
                "counterexample": "mapping_undefined"
            }
    ranks.append(rank)

mean_rank = sum(ranks) / len(ranks)
std_rank = math.sqrt(sum((x - mean_rank) ** 2 for x in ranks) / len(ranks))
support_fraction = Fraction(len([r for r in ranks if r >= n**2 * math.log(k)]), len(ranks))

conjecture_holds = support_fraction > Fraction(95, 100)
counterexample = "" if conjecture_holds else "mean_rank < n^2 log k"

return {
    "metric_name": "min_rank",
    "metric_value": mean_rank,
    "instances_tested": len(ranks),
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_rank = sum(r["metric_value"] for r in results if r["metric_value"] is not None) / len(results)
    std_rank = math.sqrt(sum((r["metric_value"] - mean_rank) ** 2 for r in results if r["metric_value"] is not None) / len(results))

```

```

support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["metric_value"] is not None for r in results):
    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_rank} std={std_rank} support_fraction={support_fraction}")
    elif support_fraction > 0.5:
        print(f"RESULT: FALSIFIED counterexample=\"mean_rank < n^2 log k\" first_failing_seed={first_failing_seed}")
    else:
        print(f"RESULT: INCONCLUSIVE support_fraction={support_fraction}")
else:
    print("RESULT: INCONCLUSIVE some trials returned None")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_524fd72d.py", line 94, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_524fd72d.py", line 62, in run_trial
    rank = min_rank_of_hodge_structure(n, k)
                                             ^
NameError: name 'k' is not defined

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed due to an undefined variable 'k', which prevented the production of data necessary to evaluate the conjecture. | next: Define the variable 'k' in the test code and rerun the test to provide evidence for or against the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12308
2	preregistration	ollama_remote	glm4:latest	0	0	5459
3	novelty	ollama_remote	glm4:latest	0	0	4886
4	novelty	ollama_remote	glm4:latest	0	0	5569
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	28068

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9755
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11595
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	31605
9	judge	ollama_remote	glm4:latest	0	0	48260

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 157505 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/5b9dbc90b6a9.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/5b9dbc90b6a9.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/5b9dbc90b6a9.tar.gz (if generated)